Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro

Lab Manual 6 (Loops in Python)

1. Introduction to Loops

1.1 What is a Loop?

A **loop** is one of the most fundamental constructs in programming. It is a control-flow structure that instructs the computer to **repeat** a block of statements multiple times — either a fixed number of times or until a certain condition is no longer satisfied.

Without loops, a programmer would have to write every repeated instruction manually. For example, printing numbers from 1 to 1,000 would require 1,000 separate `print()` statements. A loop reduces that to just three or four lines of code.

1.2 Historical Background

The concept of repeating instructions dates back to the earliest days of computing:

- **1940s — Assembly Language:** Early programmers used "jump" instructions to make the CPU return to a previous address, creating a manual loop.
- **1957 — FORTRAN:** The DO loop was introduced, allowing iteration with a counter variable — the ancestor of the modern for-loop.
- **1960s — ALGOL & COBOL:** The while/until constructs emerged, enabling condition-based iteration.
- **1991 — Python:** Guido van Rossum released Python with clean, indentation-based for and while loops that remain unchanged today.

1.3 Why Are Loops Important?

Loops are indispensable in virtually every real-world program. Here is why:

- **Efficiency:** Repeating tasks without writing duplicate code saves development time and reduces human error.

- **Data Processing:** Loops are essential for iterating over lists, files, database records, and API responses.
- **Automation:** Robotic systems, batch scripts, and AI training loops all rely on repetition at scale.
- **Algorithms:** Sorting, searching, and machine-learning gradient descent all depend on looping constructs.
- **Game Engines:** Every game runs a "game loop" — a continuous cycle that updates physics, input, and rendering 60+ times per second.

1.4 Types of Loops in Python

Loop Type	Best Used When	Key Feature
for loop	The number of iterations is known in advance	Iterates over any sequence (list, string, range)
while loop	The number of iterations depends on a condition	Keeps running as long as condition is True

Nested loops	Working with 2D data (matrices, grids)	A loop inside another loop

2. The for Loop

2.1 Syntax & Structure

The basic syntax of a for loop in Python is:

```python
for variable in sequence:
    # code block (indented by 4 spaces)
```

```
statement_1

statement_2
```

Where `variable` is a temporary name that takes the value of each element in `sequence` one at a time.

2.2 Iterating with range()

`range()` is the most common tool for generating numeric sequences in for loops.

Expression	Produces	Example
`range(n)`	0 to n-1	`range(5) → 0,1,2,3,4`
`range(a, b)`	a to b-1	`range(2,6) → 2,3,4,5`
`range(a, b, step)`	a to b-1 by step	`range(0,10,2) → 0,2,4,6,8`

2.3 Iterating Over Sequences

for loops can iterate over any iterable — strings, lists, tuples, dictionaries, and more.

Iterating Over a List

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:

print(fruit)

# Output:

# apple

# banana

# cherry
```

Iterating Over a String

```
word = "Python"

for letter in word:

print(letter, end=" ")

# Output: P y t h o n
```

Iterating Over a Dictionary

```
student = {"name": "Ali", "age": 20, "grade": "A"}

for key, value in student.items():

print(f"{key}: {value}")

# Output:

# name: Ali

# age: 20

# grade: A
```

2.4 Using enumerate() and zip()

enumerate() — Loop with Index

```
colors = ["red", "green", "blue"]
for index, color in enumerate(colors):
print(f"{index}: {color}")
# Output:
# 0: red
# 1: green
# 2: blue
```

zip() — Loop Over Two Lists Together

```
names = ["Sara", "Omar", "Zara"]
scores = [88, 95, 72]
for name, score in zip(names, scores):
print(f"{name} scored {score}")
# Output:
# Sara scored 88
# Omar scored 95
# Zara scored 72
```

2.5 Nested for Loops

A loop inside another loop. The inner loop completes all its iterations for every single iteration of the outer loop.

```
for i in range(1, 4): # Outer loop

for j in range(1, 4): # Inner loop

print(i * j, end=" ")

print() # New line after each row

# Output:

# 1 2 3

# 2 4 6

# 3 6 9
```

3. The while Loop

3.1 Syntax & Structure

```
while condition:
# Executes as long as condition is True
statement_1
statement_2
# (Must have something that eventually makes condition False)
```

⚠ Always ensure the condition eventually becomes False — otherwise the loop runs forever (infinite loop), crashing your program.

3.2 Basic while Loop Examples

Counting Up

```
count = 1
while count <= 5:
print("Count:", count)
count += 1 # Increment — moves closer to stopping condition
# Output:
```

```
# Count: 1
# Count: 2
# Count: 3
# Count: 4
# Count: 5
```

Countdown Timer

```
seconds = 5
while seconds > 0:
print(f"Launching in {seconds}...")
seconds -= 1
print("Blast off!")
# Output:
# Launching in 5...
# Launching in 4...
# ...
# Blast off!
```

User Input Validation

```
password = ""
while password != "secret123":
password = input("Enter password: ")
print("Access granted!")
```

3.3 Infinite Loops (Controlled)

Sometimes an intentional infinite loop is used — controlled with a `break` statement.

```
while True:
user_input = input("Type 'quit' to exit: ")
if user_input == "quit":
break
print(f"You typed: {user_input}")
print("Goodbye!")
```

3.4 while with else

Python's while loop supports an optional `else` block, which runs when the condition becomes False (not when broken with break).

```
num = 1

while num <= 3:

print(num)

num += 1

else:

print("Loop finished normally.")

# Output:

# 1

# 2

# 3

# Loop finished normally.
```

4. Loop Control Statements

Python provides three keywords to alter the normal flow of a loop:

Keyword	Purpose	Works In
`break`	Exit the loop immediately	for and while loops
`continue`	Skip current iteration, move to next	for and while loops
`pass`	Do nothing (placeholder statement)	for, while, if, functions

4.1 break Statement

```
for num in range(1, 11):
```

```
if num == 6:

print("Found 6! Stopping.")

break

print(num)

# Output: 1 2 3 4 5 Found 6! Stopping.
```

4.2 continue Statement

```
for num in range(1, 11):

if num % 2 == 0: # Skip even numbers

continue

print(num)

# Output: 1 3 5 7 9
```

4.3 pass Statement

```
for i in range(5):
```

```
if i == 3:

pass # Placeholder; does nothing

print(i)

# Output: 0 1 2 3 4
```

5. for vs while — When to Use Which?

Use for loop when…	Use while loop when…
You know the number of iterations in advance	Iterations depend on user input or external data
Iterating over a list, string, or dictionary	Waiting for a condition to change

Using range() to count steps	Building interactive menus or games
Traversing nested data structures	Polling sensors, network sockets, or APIs

6. Worked Problems with Solutions

Problem 1: Sum of First N Natural Numbers

Write a program to calculate the sum of all natural numbers from 1 to N using a for loop.

Code:

```
n = int(input("Enter N: "))

total = 0

for i in range(1, n + 1):

total += i

print(f"Sum of 1 to {n} = {total}")
```

Output:

```
Enter N: 10

Sum of 1 to 10 = 55
```

Problem 2: Multiplication Table

Print the multiplication table for a given number using a for loop.

Code:

```
number = int(input("Enter a number: "))
```

```
print(f"\nMultiplication Table of {number}:")

for i in range(1, 11):

print(f"{number} x {i} = {number * i}")
```

Output:

```
Enter a number: 7

Multiplication Table of 7:

7 x 1 = 7

7 x 2 = 14

...

7 x 10 = 70
```

Problem 3: Factorial Calculator

Calculate the factorial of a number using a while loop.

Code:

```
number = int(input("Enter a number: "))

factorial = 1

n = number

while n > 0:

factorial *= n

n -= 1

print(f"{number}! = {factorial}")
```

Output:

```
Enter a number: 6

6! = 720
```

Problem 4: Fibonacci Sequence

Generate the Fibonacci sequence up to N terms using a while loop.

Code:

```
n = int(input("How many terms? "))

a, b = 0, 1

count = 0

print("Fibonacci Sequence:")

while count < n:

print(a, end=" ")

a, b = b, a + b

count += 1
```

Output:

```
How many terms? 8

Fibonacci Sequence: 0 1 1 2 3 5 8 13
```

Problem 5: Prime Number Checker

Check whether a given number is prime using a for loop with break.

Code:

```
num = int(input("Enter a number: "))

is_prime = True

if num < 2:

is_prime = False

for i in range(2, int(num ** 0.5) + 1):

if num % i == 0:

is_prime = False

break

if is_prime:

print(f"{num} is a prime number.")

else:

print(f"{num} is NOT a prime number.")
```

Output:

```
Enter a number: 17

17 is a prime number.
```

Problem 6: Reverse a String

Reverse a string character-by-character using a for loop.

Code:

```
text = input("Enter a string: ")

reversed_text = ""

for char in text:

reversed_text = char + reversed_text

print(f"Reversed: {reversed_text}")
```

Output:

```
Enter a string: Python
```

```
Reversed: nohtyP
```

Problem 7: Star Pattern (Triangle)

Print a right-angled triangle pattern of stars using nested for loops.

Code:

```
rows = int(input("Enter number of rows: "))

for i in range(1, rows + 1):

for j in range(i):

print("*", end=" ")

print()
```

Output:

```
Enter number of rows: 5

*
```

```
* *

* * *

* * * *

* * * * *
```

Problem 8: Number Guessing Game

Build a simple number guessing game using a while loop.

Code:

```
import random

secret = random.randint(1, 100)

guess = 0

attempts = 0

while guess != secret:

guess = int(input("Guess the number (1-100): "))

attempts += 1

if guess < secret:

print("Too low! Try again.")

elif guess > secret:
```

```
print("Too high! Try again.")

else:

print(f"Correct! You got it in {attempts} attempts.")
```

Output:

```
Guess the number (1-100): 50

Too low! Try again.

Guess the number (1-100): 75

Too high! Try again.

Guess the number (1-100): 63

Correct! You got it in 3 attempts.
```

Problem 9: List Average Calculator

Calculate the average of a list of numbers entered by the user using a while loop.

Code:

```
numbers = []

print("Enter numbers (type 'done' to finish):")

while True:

entry = input("> ")

if entry.lower() == "done":

break

numbers.append(float(entry))

if numbers:

average = sum(numbers) / len(numbers)

print(f"Average: {average:.2f}")
```

Output:

```
Enter numbers (type 'done' to finish):

> 10

> 20

> 30

> done

Average: 20.00
```

Problem 10: FizzBuzz

Print numbers 1–50. For multiples of 3 print Fizz, for multiples of 5 print Buzz, for both print FizzBuzz.

Code:

```
for i in range(1, 51):
if i % 15 == 0:
print("FizzBuzz")
elif i % 3 == 0:
print("Fizz")
elif i % 5 == 0:
print("Buzz")
else:
print(i)
```

Output:

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
...
```

7. Practice Tasks — 20 Exercises

Instructions: Complete each task on your own. Write the code in your Python IDE and test it. No solutions are provided — challenge yourself!

Task 01: Even Numbers List

Use a for loop to print all even numbers between 1 and 100 on a single line, separated by spaces.

My Solution:

Task 02: Sum of Digits

Write a program that accepts an integer and calculates the sum of its digits using a while loop. (e.g., 1234 → 10)

My Solution:

Task 03: Countdown Timer

Print a countdown from 100 to 1 using a for loop with a step of -1. On the last line print 'Time is up!'.

My Solution:

Task 04: ATM PIN Lock

Simulate an ATM that allows a maximum of 3 PIN attempts. Use a while loop. After 3 wrong attempts, print 'Card blocked!'.

My Solution:

Task 05: Palindrome Checker

Ask the user to enter a word. Use a for loop to check if it is a palindrome (reads the same forwards and backwards).

My Solution:

Task 06: Pyramid Pattern

Print a pyramid of numbers using nested for loops:

1

123

12345

My Solution:

Task 07: Grade Calculator

Accept 5 test scores from the user (using a for loop) and calculate the average. Print a letter grade based on standard grading (A/B/C/D/F).

My Solution:

Task 08: List Flattener

Given a nested list [[1,2],[3,4],[5,6]], use nested for loops to print all values on a single line.

My Solution:

Task 09: Shopping Cart Total

Allow the user to keep entering item prices (while loop) until they type 0. Print the total cost and the number of items purchased.

My Solution:

Task 10: Prime Numbers in Range

Print all prime numbers between two user-entered values using a for loop and an inner while loop for divisibility checks.

My Solution:

Task 11: Pascal's Triangle

Generate and print the first 6 rows of Pascal's Triangle using nested loops and a list to track values.

My Solution:

Task 12: String Frequency Counter

Count how many times each character appears in a user-inputted string using a for loop and a dictionary.

My Solution:

Task 13: Multiplication Table Grid

Print a 10x10 multiplication table grid using nested for loops, with headers and aligned columns.

My Solution:

Task 14: Collatz Conjecture

For a user-given number, apply the Collatz sequence (if even: n/2; if odd: 3n+1) using a while loop until reaching 1. Count the steps.

My Solution:

Task 15: Word Reverser

Accept a sentence from the user and print each word in reverse order (not the characters, but the order of words in the sentence).

My Solution:

Task 16: Number Pattern — Floyd's Triangle

Print Floyd's Triangle up to 5 rows (a right-angled triangle of sequential integers starting from 1).

My Solution:

Task 17: Login System

Build a simple login system with a username ('admin') and password ('pass123'). Allow 3 attempts using a while loop. Lock the account on failure.

My Solution:

Task 18: Powers of 2

Use a while loop to print all powers of 2 that are less than 10,000 (i.e., 1, 2, 4, 8, 16 … up to the last value below 10,000).

My Solution:

Task 19: List Duplicate Remover

Given a list with duplicates (you define it), use a for loop to create a new list with only unique values, preserving the original order.

My Solution:

Task 20: Mini Calculator (Loop-Based Menu)

Build a loop-based menu calculator. Show options: Add, Subtract, Multiply, Divide, Quit. Keep running until Quit is selected. Handle division by zero.

My Solution:

8. Quick Reference Cheat Sheet

for Loop Cheat Sheet

```
# Basic for loop

for i in range(10):

print(i)

# Loop with start and step

for i in range(0, 20, 2):

print(i)

# Loop over list

for item in my_list:
```

```
print(item)

# Loop with index

for i, val in enumerate(my_list):

print(i, val)

# Nested loops

for i in range(3):

for j in range(3):

print(i, j)
```

while Loop Cheat Sheet

```
# Basic while

i = 0

while i < 10:

print(i)

i += 1

# While with break

while True:

x = input("Enter: ")

if x == "q":
```

```
break

# While with else

n = 5

while n > 0:

n -= 1

else:

print("Done")
```

Control Statements Cheat Sheet

```
# break — exit loop immediately

for i in range(10):

if i == 5:

break

# continue — skip this iteration

for i in range(10):

if i % 2 == 0:

continue

print(i)

# pass — do nothing (placeholder)
```

```
for i in range(10):

pass
```

End of Lab Manual — Python Loops